

<

Cours Modélisation et vérification des systèmes informatiques
Exercices
Modélisation d'algorithmes en PlusCal (I)
par Dominique Méry
8 octobre 2026

Exercice 1 *appex4...*

Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA contenant une expression de l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence d'erreurs à l'exécution.

Question 1.1 *appex4_1_1.tla*

$$\begin{aligned} \ell_1 : x = 10 \wedge y = z+x \wedge z = 2 \cdot x \\ y := z+x \\ \ell_2 : x = 10 \wedge y = x+2 \cdot 10 \end{aligned}$$
Question 1.2 *On suppose que p est un nombre premier :*

$$\begin{aligned} \ell_1 : x = 2^p \wedge y = 2^{p+1} \wedge x \cdot y = 2^{2 \cdot p+1} \\ x := y+x+2^x \\ \ell_2 : x = 5 \cdot 2^p \wedge y = 2^{p+1} \end{aligned}$$
Question 1.3

$$\begin{aligned} \ell_1 : x = 1 \wedge y = 12 \\ x := 2 \cdot y \\ \ell_2 : x = 1 \wedge y = 24 \end{aligned}$$
Question 1.4

$$\begin{aligned} \ell_1 : x = 11 \wedge y = 13 \\ z := x; x := y; y := z; \\ \ell_2 : x = 26/2 \wedge y = 33/3 \end{aligned}$$

Question 1.5 Dans cette question, on considère l'algorithme correspondant au calcul du maximum de deux nombres.

Exercice 2 Annoter chaque algorithme et le traduire avec la traduction de PlusCal en TLA⁺.

Question 2.1 *pluscal_loop*

Listing 1 – Fragment

```

1      variables x=x0;
2      {
3      while (0<x)
4      {
5          x := x - 1;
6      };
7      }
8      }
```

Question 2.2 *pluscal_max*

Requires : $x_0, y_0 \in \mathbb{N}$
Ensurees : $z_f = \max(x_0, y_0)$

```

 $\ell_0 : \{\dots\}$ 
if  $X < Y$  then
  |  $\ell_1 : \{\dots\}$ 
  |  $Z := Y;$ 
  |  $\ell_2 : \{\dots\}$ 
else
  |  $\ell_3 : \{\dots\}$ 
  |  $Z := X;$ 
  |  $\ell_4 : \{\dots\}$ 
  |
;
 $\ell_5 : \{\dots\}$ 

```

Algorithme 1: *MAX* annotée

Listing 2 – FRagment

```

1  variables x=a, y = b, z;
2  {
3    if ( x < y ) {
4      z:=y;
5    }
6  else
7  {
8    z:= x;
9  };
10 }
11 }

```

Exercice 3

On considère l'algorithme suivant :

2) **START**
 $\{x_1 \geq 0 \wedge x_2 > 0\}$
 $(y_1, y_2, y_3) \leftarrow (x_1, 0, x_2);$
while $y_3 \leq y_1$ **do** $y_3 \leftarrow 2y_3;$
while $y_3 \neq x_2$ **do**
 begin $(y_2, y_3) \leftarrow (2y_2, y_3/2);$
 if $y_3 \leq y_1$ **do** $(y_1, y_2) \leftarrow (y_1 - y_3, y_2 + 1)$
 end;
 $(z_1, z_2) \leftarrow (y_1, y_2)$
 $\{0 \leq z_1 < x_2 \wedge x_1 = z_2 x_2 + z_1\}$
HALT

Question 3.1 Montrer que cet algorithme est apriiellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du langage PlusCal.

Question 3.2 Montrer qu'il est sans erreur à l'exécution.

Exercice 4 ✓

On considère l'algorithme suivant :

```

START
{ $x_1 > 0 \wedge x_2 > 0$ }
( $y_1, y_2, y_3, y_4$ )  $\leftarrow$  ( $x_1, x_2, x_2, 0$ );
while  $y_1 \neq y_2$  do
  begin while  $y_1 > y_2$  do ( $y_1, y_4$ )  $\leftarrow$  ( $y_1 - y_2, y_4 + y_3$ );
  end; while  $y_2 > y_1$  do ( $y_2, y_3$ )  $\leftarrow$  ( $y_2 - y_1, y_3 + y_4$ )
  end;
( $z_1, z_2$ )  $\leftarrow$  ( $y_1, y_3 + y_4$ )
{ $z_1 = \text{gcd}(x_1, x_2) \wedge z_2 = \text{scm}(x_1, x_2)$ }
HALT

```

where $\text{scm}(x_1, x_2)$ is the smallest common multiple of x_1 and x_2 ; that is, it is their product divided by the greatest common divisor.

Question 4.1 Montrer que cet algorithme est aptriellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du langage PlusCal.

Question 4.2 Montrer qu'il est sans erreur à l'exécution.

◇ **Solution de l'exercice 4**

Fin 4

Exercice 5 ✓

Nous allons utiliser la fonctionnalité de traduction d'un algorithme PlusCal en un module TLA pour vérifier des algorithmes. Pour chaque question, on écrira un module TLA contenant une expression de l'algorithme puis on traduira par un module et ensuite on analysera le module obtenu par rapport à la correction partielle et l'absence d'erreurs à l'exécution.

Question 5.1 Soit l'annotation suivante :

$$\begin{aligned} \ell_1 : & x = 10 \wedge y = z + x \wedge z = 2 \cdot x \\ & y := z + x \\ \ell_2 : & x = 10 \wedge y = x + 2 \cdot 10 \end{aligned}$$

Traduire en PlusCal.

Exercice 6 On considère l'algorithme suivant :

```

START
{ $x_1 > 0 \wedge x_2 > 0$ }
 $(y_1, y_2, y_3, y_4) \leftarrow (x_1, x_2, x_2, 0)$ ;
while  $y_1 \neq y_2$  do
  begin while  $y_1 > y_2$  do  $(y_1, y_4) \leftarrow (y_1 - y_2, y_4 + y_3)$ ;
  end; while  $y_2 > y_1$  do  $(y_2, y_3) \leftarrow (y_2 - y_1, y_3 + y_4)$ 
end;
 $(z_1, z_2) \leftarrow (y_1, y_3 + y_4)$ 
 $\{z_1 = \gcd(x_1, x_2) \wedge z_2 = \text{scm}(x_1, x_2)\}$ 
HALT

```

where $\text{scm}(x_1, x_2)$ is the smallest common multiple of x_1 and x_2 ; that is, it is their product divided by the greatest common divisor.

Question 6.1 Montrer que cet algorithme est aptriellement correct par rapport à sa précondition et à sa postcondition qu'il faudra énoncer. Pour cela, on traduira cet algorithme sous forme d'un module à partir du lanage PlusCal.

Question 6.2 Montrer qu'il est sans erreur à l'exécution.

Exercice 7 Exponentiation en PlusCal

Ecrire l'algorithme de l'exponentiation avec PlusCal.

On suppose que x_1 et x_2 sont des constantes.

```

precondition :  $x_1 \in \mathbb{N} \wedge x_2 \in \mathbb{N} \wedge x_1 \neq 0$ 
postcondition :  $z = x_1^{x_2}$ 
local variables :  $y_1, y_2, y_3 \in \mathbb{Z}$ 

 $\ell_0 : \{y_1, y_2, y_3, z \in \mathbb{Z}\}$ 
 $(y_1, y_2, y_3) := (x_1, x_2, 1)$ ;
 $\ell_1 : \{y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
while  $y_2 \neq 0$  do
   $\ell_2 : \{y_2 \neq 0 \wedge y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
  if  $\text{impair}(y_2)$  then
     $\ell_3 : \{\text{impair}(y_2) \wedge y_2 \neq 0 \wedge \boxed{\dots} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
     $y_2 := y_2 - 1$ ;
     $\ell_4 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge y_3 \cdot y_1 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
     $y_3 := y_3 \cdot y_1$ ;
     $\ell_5 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
  ;
   $\ell_6 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge \boxed{\dots} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
   $y_1 := y_1 \cdot y_1$ ;
   $\ell_7 : \{y_2 \geq 0 \wedge \text{pair}(y_2) \wedge y_3 \cdot y_1^{y_2 \text{ div } 2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
   $y_2 := y_2 \text{ div } 2$ ;
   $\ell_8 : \{y_2 \geq 0 \wedge \boxed{\dots} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
;
 $\ell_9 : \{y_2 = 0 \wedge \boxed{\dots} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z}\}$ 
 $z := y_3$ ;
 $\ell_{10} : \{y_2 = 0 \wedge y_3 \cdot y_1^{y_2} = x_1^{x_2} \wedge y_1, y_2, y_3 \in \mathbb{N} \wedge z \in \mathbb{Z} \wedge z = x_1^{x_2}\}$ 

```

Algorithme 2: Algorithme de l'exponentiation indienne annoté

Exercice 8 Vérifier le programme C suivant en la traduisant dans un module PlusCal. Pour cela, on conservera les éléments algorithmiques suffisants pour analyser le programme C. Par exemple, on ne traduira pas les instructions de lecture ou les inclusions de bibliothèques.

```

1
2
3  /* C Program to find roots of a quadratic equation when coefficients are
   entered by user. */
4  /* Library function sqrt() computes the square root. */
5
6  #include <stdio.h>
7  #include <math.h> /* This is needed to use sqrt() function.*/
8  int main()
9  {
10     float a, b, c, determinant, r1,r2, real, imag;
11     printf("Enter coefficients a, b and c: ");
12     scanf("%f%f%f", &a, &b, &c);
13     determinant=b*b-4*a*c;
14     if (determinant>0)
15     {
16         r1= (-b+sqrt(determinant))/(2*a);
17         r2= (-b-sqrt(determinant))/(2*a);
18         printf("Roots are: %.2f and %.2f", r1 , r2);
19     }
20     else if (determinant==0)
21     {
22         r1 = r2 = -b/(2*a);
23         printf("Roots are: %.2f and %.2f", r1, r2);
24     }
25     else
26     {
27         real= -b/(2*a);
28         imag = sqrt(-determinant)/(2*a);
29         printf("Roots are: %.2f+%.2fi and %.2f-%.2fi", real, imag, real, imag);
30     }
31     return 0;
32 }
33
34 Output 1
35
36
37 Enter coefficients a, b and c: 2.3
38 4
39 5.6
40 Roots are: -0.87+1.30i and -0.87-1.30i
41
42 Output 2
43
44
45 Enter coefficients a, b and c: 4
46 1
47 0
48 Roots are: 0.00 and -0.25
49
50 To solve this program, library function sqrt() is used. This function
   calculates the square root of a number. Learn more about, sqrt() function.
51 Similar C Programming Examples
52 C Program to Add Two Complex Numbers by Passing Structure to a Function
53 C Program to Find Quotient and Remainder of Two Integers Entered by User
54 C Program to Check Prime or Armstrong Number Using User-defined Function
55 C Program to Display Prime Numbers Between Intervals Using User-defined

```

Function
56 C Program to Display its own Source Code as Output

Exercice 9

Vérifier le programme C suivant en la traduisant dans un module PlusCal.

```
1  /* C program to check whether a number is palindrome or not */
2
3  #include <stdio.h>
4  int main()
5  {
6      int n, reverse=0, rem,temp;
7      printf("Enter_an_integer:_");
8      scanf("%d", &n);
9      temp=n;
10     while(temp!=0)
11     {
12         rem=temp%10;
13         reverse=reverse*10+rem;
14         temp/=10;
15     }
16     /* Checking if number entered by user and it's reverse number is equal. */
17     if(reverse==n)
18         printf("%d_is_a_palindrome.",n);
19     else
20         printf("%d_is_not_a_palindrome.",n);
21     return 0;
22 }
```

◇ Solution de l'exercice 9

Fin 9

Exercice 10 Vérifier le programme C suivant en la traduisant dans un module PlusCal.

```
1  #include <stdio.h>
2  int main()
3  {
4      int n,count=0;
5      printf("Enter_an_integer:_");
6      scanf("%d", &n);
7      while(n!=0)
8      {
9          n/=10;          /* n=n/10 */
10         ++count;
11     }
12     printf("Number_of_digits:_%d",count);
13 }
```

Exercice 11 Nous avons récupéré un programme C qui réalise le tri à bulles. Traduire ce programme en PlusCal et analyser sa correction partielle et l'absence d'erreurs à l'exécution.

```
1
2
3  void tri_a_bulle(int* t, int const size)
4  {
5      int en_desordre = 1;
6      int i,j;
7
8      for (i = 0; (i < size) && en_desordre; ++i)
```

```

9      {
10          en_desordre = 0;
11          for (j = 1; j < (size - i); ++j)
12          {
13              if (t[j-1] > t[j])
14              {
15                  int temp = t[j-1];
16                  t[j-1] = t[j];
17                  t[j] = temp;
18                  en_desordre = 1;
19              }
20          }
21      }
22  }

```

```

1      #include<algorithm> // pour la fonction swap
2
3      void tri_a_bulle(int *t, int const size)
4      {
5          bool en_desordre = true;
6          for (int i = 0; i < size && en_desordre; ++i)
7          {
8              en_desordre = false;
9              for (int j = 1; j < size - i; ++j)
10                 if (t[j-1] > t[j])
11                 {
12                     std::swap(t[j], t[j-1]);
13                     en_desordre = true;
14                 }
15          }
16      }

```

Exercice 12 Analyser le programme C suivant qui réalise le tri par sélection.

```

1      void selection(int *t, int taille)
2      {
3          int i, mini, j, x;
4
5          for (i = 0; i < taille - 1; i++)
6          {
7              mini = i;
8              for (j = i + 1; j < taille; j++)
9                  if (t[j] < t[mini])
10                     mini = j;
11              x = t[i];
12              t[i] = t[mini];
13              t[mini] = x;
14          }
15      }

```

Exercice 13 GCD en Event-B

Soit l'algorithme GCD calculant le pgcd de deux nombres entiers positifs quelconques a et b.

```

begin
start:
  while  $y_1 \neq y_2$  do
    loop:
      if  $y_1 < y_2$  then
        inloop1:
           $y_2 := y_2 - y_1$ ;
        endif1:
      else
        inloop2:
           $y_1 := y_1 - y_2$ ;
        endif2:
      fi
    endif
  end-while
endloop:
 $g := y_1$ ;
end

```

Question 13.1 Traduire l'algorithme en un module TLA^+ qui précise les annotations et qui précise les invariants pour la correction partielle et l'absence d'erreurs à m'exécution.

Question 13.2 Construire une machine Event-B qui liste les actions de l'algorithme et qui donne l'invariant pour la correction partielle et l'absence d'erreurs à l'exécution.